

```

{
  "id": "1",
  "title": "Apple iPhone XR",
  "displayName": "Apple iPhone XR",
  "shortDesc": "iPhone XR features the most advanced LCD in a smartphone—a 6.1-inch Liquid Retina display",
  "longDesc": "iPhone XR features the most advanced LCD in a smartphone—a 6.1-inch Liquid Retina display",
  "listPrice": 749.99,
  "salePrice": 9,
  "upc": "133131",
  "mpn": "13131",
  "reviews": [
    {
      "id": "1",
      "reviewTitle": "I love everything about this phone",
      "reviewText": "So far I have had no issues at all with this phone",
      "submittedBy": "Siva",
      "rating": 4
    }
  ],
  "techSpecs": [
    {
      "id": "1",
      "key": "Camera",
      "value": "12MP Camera with OIS & 7MP TrueDepth Front Camera"
    },
    {
      "id": "3",
      "key": "Carriers",
      "value": "AT&T,Sprint,Verizon"
    },
    {
      "id": "2",
      "key": "Capacity",
      "value": "64GB"
    }
  ]
}

```

Figure 5: REST in action

GraphiQL

Prettify History

```

1 # Welcome to GraphiQL
2 #
3 # GraphiQL is an in-browser tool for writing, validating, and
4 # testing GraphQL queries.
5 #
6 # The intention of the below query is to get only product id, title, short description
7 # and list price, AND reviews(Though review object has review id, review Title,
8 # review Text, Submitted By and rating. We want ONLY review title AND review text)
9 # of the product.
10
11 {
12   product(id:"1") {
13     id,
14     title,
15     shortDesc,
16     listPrice,
17     reviews {
18       reviewTitle,
19       reviewText
20     }
21   }
22 }

```

Figure 6: GraphiQL in action

to existing APIs, we need to be extra vigilant as the changes might break the clients. To avoid that, we do versioning of APIs as and when we plan to release changes to existing APIs. When we introduce new versions, we need to manage more APIs (i.e., more compute

power and more manpower), and also have to plan how to deprecate older versions. Discipline and communication are needed when we have more versions of an API. With REST, we cannot do silent releases.

There is another solution called GraphQL that can help here. We will

look at how GraphQL addresses the issues discussed in an upcoming article. Meanwhile, let us check out the request and response paradigm with GraphQL, and how the latter makes clients happy by serving them what they want.

Some entities that have adopted GraphQL are listed at <https://graphql.org/users/>.

I have created a sample application to showcase the differences between REST and GraphQL. First, we will look at the REST implementation of a simple product detail end point. I have used Spring Boot to demonstrate REST. You can download the sample project from https://github.com/2013techsmarts/Code_Samples/tree/master/rest-demo, and follow the steps outlined in the README (https://github.com/2013techsmarts/Code_Samples/blob/master/README.md) to set up the project. I am not discussing setup details here as it is beyond the scope of this article. Assuming that your project is up and running, make a call to http://localhost:8080/product/{product_id} endpoint to get product detail JSON, as shown in Figure 5.

You may have observed that JSON delivers the entire product JSON, including reviews and technical specifications, though we are not interested in all the elements of a given product.

Let us now look at GraphQL in action by getting product details in a selective manner. To demonstrate GraphQL, I again used Spring Boot. You can download the sample project from https://github.com/2013techsmarts/Code_Samples/tree/master/graphql-demo, and follow the steps outlined in the README (https://github.com/2013techsmarts/Code_Samples/blob/master/README.md) to set up the project. I am not discussing setup details here as that is beyond the scope of this article.

I am assuming that your project is up and running so you can see GraphQL in action, in which case I am interested